MOBL-05: User Action Tracking

> **Series:** MOBL — Mobile Monitoring | **Notebook:** 5 of 12 | **Created:** February 2026 | **Last Updated:** 11/11/2026

Overview

User actions are the foundation of mobile Real User Monitoring (RUM) in Dynatrace. Every tap, swipe, app launch, and custom interaction is captured as a **user action** — a discrete, measurable event that reveals how users interact with your mobile application. This notebook explores how Dynatrace captures these interactions, from auto-detected taps and navigation events to custom actions you instrument yourself. You will also learn about rage tap detection (a key frustration signal) and how to query action data with DQL for performance analysis and optimization.

Table of Contents

1. What Are User Actions?
2. Auto-Detected Actions
3. Action Lifecycle
4. Rage Tap Detection
5. Custom User Actions
6. Querying Actions with DQL
7. Optimizing Action Naming

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
Mobile App	At least one mobile app with the Dynatrace Mobile SDK configured
Permissions	`storage:user.events:read`, `storage:user.sessions:read`
Data	User action data actively flowing from mobile devices
Prior Notebooks	Completed MOBL-01 through MOBL-04 recommended

1. What Are User Actions?

A **user action** represents a single, discrete user interaction with a mobile application. Dynatrace captures each action with rich context including the action name, type, duration, associated network requests, and any errors that occurred during the interaction.

Action Properties

Every user action includes the following key properties:

Property	Description	Example
Name	Human-readable label describing the interaction	`"Tap on Login"`, `"Add to Cart"`
Type	Category of interaction (`interaction.type` in Grail)	a tap, a swipe, an app start, a custom action
Duration	Time from action start to completion (including child events)	`450ms`
Network Requests	HTTP calls triggered during the action	API calls, image loads
Errors	Any crashes or HTTP errors during the action	500 responses, exceptions

How Actions Appear in Grail

In Grail, mobile interactions are `user.events` records typed by `characteristics.*` flags rather than by a single action-type field:

Interaction	Grail representation	Detection
User action (tap or other interaction plus the requests it triggers)	`characteristics.has_user_action`; control in `ui_element.detected_name`, kind in `interaction.type`	Auto-detected for supported UI components
User interaction (standalone tap, click, swipe)	`characteristics.has_user_interaction`	Auto-detected
App start (cold, warm, hot)	`characteristics.has_app_start`	Always auto-detected
Custom action	`characteristics.has_user_action` with `characteristics.is_api_reported`	Manual instrumentation (`enterAction`)
Rage tap	no dedicated field in the semantic dictionary (read 09/28/2026)	Detected by OneAgent (Section 4)

> **Note:** The exact set of auto-detected action types varies by platform and UI framework. See the next section for a detailed compatibility matrix.

2. Auto-Detected Actions

> **New (OneAgent for Mobile 8.347 — released 08/28/2026, rollout from 09/08/2026): every launch produces an app-start action.** The agent now tracks *"an app start user action on every app launch"*, spanning initialization through the first fully settled screen, and it can be waterfall-visualized and filtered by launch type. That is a new member of the auto-detected population below — expect action counts

> to rise once instrumented builds ship, and check any dashboard that counts actions per session before reading the change as a regression.
>
> The same release **caps inflated cold-start durations** — verbatim, it *"caps inflated cold app-start durations so you see realistic warm-start timings"*. Startup-duration percentiles will shift downward as a result. That is a measurement correction, not an improvement in the app, so re-baseline startup SLOs and alerts rather than reporting a win.
>
> Mobile agent versions land with **app releases, not tenant updates** (MOBL-01), so this arrives across your user base at the pace of store adoption.

Dynatrace auto-instrumentation covers both imperative (UIKit, Android Views) and declarative (SwiftUI, Jetpack Compose) UI frameworks, through different mechanisms: runtime or bytecode instrumentation for UIKit and Views, the Android Gradle plugin for Compose (on by default from plugin 8.271), and the build-time SwiftUI instrumentor (`DTSwiftInstrumentor`) for SwiftUI.

Platform Compatibility Matrix

Action	iOS (UIKit)	iOS (SwiftUI)	Android (Views)	Android (Compose)
Button tap	Auto	Auto (DTSwiftInstrumentor)	Auto	Auto (plugin 8.271+)
List item tap	Auto	Auto (DTSwiftInstrumentor, 8.265+)	Auto	Auto (plugin 8.271+, `clickable` modifiers)
Navigation	Auto	Auto (DTSwiftInstrumentor, `NavigationLink` 8.265+)	Auto	Auto (plugin 8.271+)
App start	Auto	Auto	Auto	Auto
App background/foreground	Auto	Auto	Auto	Auto

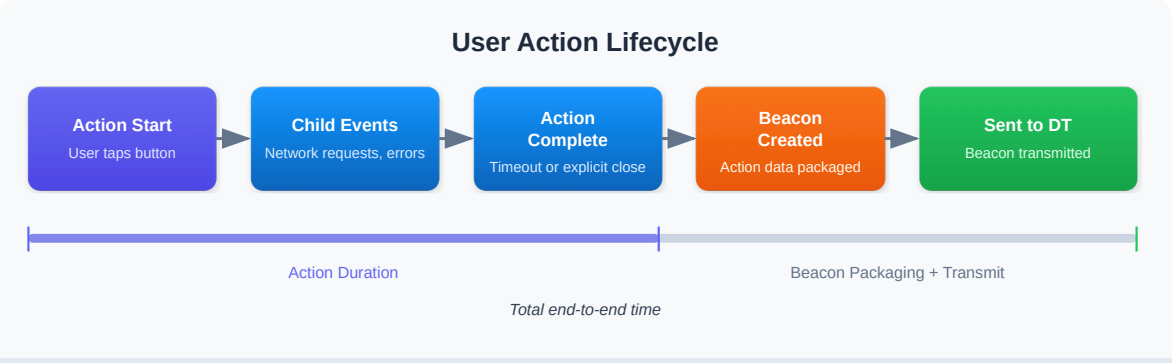
Key takeaways:

- **UIKit and Android Views** provide the most complete auto-detection out of the box. Button taps, list selections, and navigation transitions are captured automatically.
- **SwiftUI and Jetpack Compose** are auto-instrumented too, but by a build step: the SwiftUI instrumentor for SwiftUI (MOBL-02 §5) and the Android Gradle plugin for Compose (MOBL-03 §5). If that step is missing from the build, their interactions go uncaptured.
- **App start and lifecycle events** (background/foreground) are always auto-detected regardless of UI framework.

> **Tip:** If your app uses SwiftUI, add `DTSwiftInstrumentor` to the build (and CI) early; for Compose, confirm the plugin is 8.271 or later. Reserve custom actions (Section 5) for business-level flows.
>
> **Sources:** What's new in OneAgent for Mobile 8.347 (DT docs) — the app-start user action and cold-start capping quoted above; Instrument Android apps (DT docs) — *"Jetpack Compose auto-instrumentation is enabled by default starting with Dynatrace Android Gradle plugin version 8.271."*; Instrument SwiftUI controls (DT docs).

3. Action Lifecycle

Understanding the lifecycle of a user action is critical for interpreting duration metrics and troubleshooting slow interactions.



Each user action captures timing, child events, and context before being sent as a beacon.

Lifecycle Phases

1. **Action Start** — The SDK detects a user interaction (tap, swipe) or the developer calls `enterAction()`. A timer begins.
2. **Child Event Association** — Any network requests, web API calls, or errors that occur while the action is open are automatically linked as child events. This gives you full visibility into what happened *during* the interaction.
3. **Action End** — The action closes in one of three ways:
   - **Auto-close:** The SDK detects the interaction is complete (e.g., the UI finished loading).
   - **Manual close:** The developer calls `leaveAction()` on a custom action.
   - **Timeout:** If no new child events arrive within the agent's timeout window, the action closes automatically. The window differs by platform and framework (the React Native plugin documents a fixed 1000 ms for its auto actions on Android), so check your agent's documentation rather than assuming a value.
4. **Beacon Transmission** — Completed actions are batched into a beacon and sent to Dynatrace during the next transmission cycle.

Duration Calculation

The action duration is measured from **Action Start** to **Action End**. It includes the time for all child events to complete. This means a tap action that triggers a slow API call will have a longer duration than a tap that only updates the local UI.

> **Important:** The action timeout directly affects duration measurement. If the timeout is set too long, actions will appear artificially slow. If set too short, child events may not be properly associated.

4. Rage Tap Detection

Rage taps occur when a user rapidly and repeatedly taps the same area of the screen. This behavior is a strong signal of **user frustration** — typically caused by unresponsive UI elements, slow loading, or confusing interaction patterns.

How Dynatrace Detects Rage Taps

OneAgent detects rage taps automatically (Android Gradle plugin 8.231+). On Android it can monitor only touch events handled by an `Activity`; components with their own touch processing, such as `Dialog` and `DreamService`, are not covered. Detection is on by default; switch it off per configuration with the `behavioralEvents` block:

```
configure&lt;com.dynatrace.tools.android.dsl.DynatraceExtension&gt; {
    configurations {
        create("sampleConfig") {
            behavioralEvents {
                detectRageTaps(false)
            }
        }
    }
}
```

> **Correction (09/28/2026).** Earlier revisions gave detection defaults (3 taps, 1 second, a proximity radius), a tunable sensitivity setting and a `RageTap` user-action type. None of those is documented, and the semantic dictionary (read 09/28/2026) has no rage-tap field on `user.events`. §6.3 therefore uses a clearly labelled query-side heuristic.
>
> **Sources:** Configure monitoring capabilities (DT docs) — *"OneAgent detects such behavior as a rage tap."*

Why Rage Taps Matter

Rage taps are a leading indicator of poor user experience. Common root causes include:

- **Unresponsive buttons** — The UI does not provide feedback that the tap was registered.
- **Slow transitions** — Navigation takes so long that users tap again thinking the first tap failed.
- **Broken interactions** — The tap target is not wired to any action (dead zone).
- **Layout shifts** — UI elements move after rendering, causing users to miss their intended target.

> **Tip:** Combine rage tap analysis with crash and error data to prioritize UX fixes that have the biggest impact on user satisfaction.

5. Custom User Actions

Custom user actions let you measure business-critical interactions that are not automatically captured by the SDK. Use them to wrap multi-step processes like checkout flows, search operations, or any logic where you want precise timing and child event association.

> **Sources:** OneAgent SDK for iOS (DT docs), OneAgent SDK for Android (DT docs), dynatrace_flutter_plugin (pub.dev).

iOS (Swift)

```
// iOS (Swift)
let action = DTXAction.enter(withName: "Add to Cart")
// ... perform the action logic (API calls, UI updates) ...
action?.leave()   // enter(withName:) returns an optional
```

Android (Kotlin)

```
// Android (Kotlin)
val action = Dynatrace.enterAction("Add to Cart")
// ... perform the action logic (API calls, UI updates) ...
action.leaveAction()
```

Flutter (Dart)

```
// Flutter (Dart)
DynatraceRootAction action = Dynatrace().enterAction('Add to Cart');
// ... perform the action logic (API calls, UI updates) ...
action.leaveAction();
```

> **Sources:** OneAgent SDK for iOS (DT docs) — the Swift custom-action sample closes the action with `action?.leave()`; dynatrace_flutter_plugin (pub.dev) — *"Only one sub-action level is allowed; you can't create a sub sub-action."*

Best Practices for Custom Actions

Practice	Description
Always call leave/leaveAction	Forgetting to close an action causes it to time out, inflating duration metrics
Use descriptive names	`"Add to Cart"` is better than `"button_click_42"`
Avoid nesting too deeply	Keep hierarchies shallow — the Flutter plugin allows only one sub-action level
Wrap error handling	Ensure `leaveAction()` is called in both success and error paths
Report values	Use `action.reportValue()` / `action.reportEvent()` on the open action to attach business context -- reported values must be part of a user action

6. Querying Actions with DQL

Mobile user-action data is stored in the Grail **`user.events`** store, one record per event, with `characteristics.has_user_action` marking user actions and `in(dt.rum.agent.type, {"android", "ios"})` separating mobile from web events (MOBL-10 §1). Session-level data (one record per session) is in **`user.sessions`**. The useful action fields are `frontend.name` (the app), `ui_element.detected_name` (the control), `interaction.type`, and `duration` (a duration value, so compare it with `duration > 2s`, not a number).

> **Correction (09/28/2026).** Earlier revisions queried mobile actions from `bizevents` with `event.provider == "www.dynatrace.com/mobile"` and `useraction.*` fields. That data model does not exist: every such query ran without error and returned zero

> rows. All queries in this notebook now read `user.events`. `sendBizEvent()` payloads (MOBL-12 §2) are the one mobile data type that does land in `bizevents`.

6.1 Recent User Actions

Retrieve the most recent user actions across all monitored mobile applications to see what users are doing right now.

```
// Recent user actions across all mobile apps
fetch user.events, from:-1h
| filter in(dt.rum.agent.type, {"android", "ios"}) and
characteristics.has_user_action
| fields start_time, frontend.name, ui_element.detected_name,
interaction.type, duration
| sort start_time desc
| limit 50
```

6.2 Action Volume by Interaction Type

Understand the distribution of interaction types to see which interaction patterns dominate your mobile app usage.

```
// Action volume by interaction type across all apps
fetch user.events, from:-1h
| filter in(dt.rum.agent.type, {"android", "ios"}) and
characteristics.has_user_action
| summarize action_count = count(), by:{interaction.type}
| sort action_count desc
```

6.3 Repeated Taps on One Element

Find sessions in which a user tapped the same element five or more times in 24 hours. This is a query-side frustration heuristic built on `characteristics.has_user_interaction`, not the agent's own rage-tap detection. Tune the threshold to your app.

```
// Repeated taps on the same element within one session (frustration
heuristic)
// This is a query-side heuristic, NOT Dynatrace's rage-tap detection: the
semantic
// dictionary (read 09/28/2026) has no rage-tap field on user.events.
fetch user.events, from:-24h
| filter in(dt.rum.agent.type, {"android", "ios"}) and
characteristics.has_user_interaction
| summarize taps = count(), by:{dt.rum.session.id, frontend.name,
ui_element.detected_name}
| filter taps &gt;= 5
| sort taps desc
| limit 50
```

6.4 Action Trends Over Time

Visualize user action trends on an hourly basis, split by interaction type. This query produces a time-series chart suitable for dashboards.

```
// User action trends over time (hourly)
fetch user.events, from:-24h
| filter in(dt.rum.agent.type, {"android", "ios"}) and
characteristics.has_user_action
| makeTimeseries action_count = count(), by:{interaction.type}, interval:1h
```

6.5 Top Apps by Action Volume

Rank your mobile applications by the total number of user actions to identify the most actively used apps.

```
// Top apps by action volume
fetch user.events, from:-1h
| filter in(dt.rum.agent.type, {"android", "ios"}) and
characteristics.has_user_action
| summarize action_count = count(), by:{frontend.name}
| sort action_count desc
| limit 10
```

7. Optimizing Action Naming

Poorly named user actions make analysis difficult and can cause high-cardinality issues in aggregation queries. Follow these best practices to keep action names clean and useful.

Naming Rules

Rule	Good Example	Bad Example	Why
Use descriptive, stable names	`"Add to Cart"`	`"btn_click"`	Descriptive names are self-documenting in queries and dashboards
Avoid dynamic values	`"View Product Details"`	`"View Product #48291"`	Dynamic values create thousands of unique action names, making aggregation impossible
No user IDs in names	`"User Profile"`	`"Profile: user_abc123"`	User-specific names are a cardinality explosion and a potential privacy issue
No timestamps in names	`"Refresh Dashboard"`	`"Refresh 2026-02-24T10:30"`	Timestamps guarantee every action name is unique, defeating grouping
Use consistent casing	`"Search Products"`	`"search products"` / `"SEARCH_PRODUCTS"`	Inconsistent casing creates duplicate entries in aggregations
Group related actions	`"Cart: Add Item"`, `"Cart: Remove Item"`	`"addToCart"`, `"removeFromBasket"`	Prefixes help group related actions in sorted lists

Action Naming Strategy

Adopt a naming convention across your team and enforce it through code review:

1. **Use a prefix for the feature area** — `"Cart: "`, `"Search: "`, `"Profile: "`, `"Checkout: "`

2. **Describe the user intent** — `"Add Item"`, `"Apply Filter"`, `"Submit Order"`
3. **Combine into a consistent pattern** — `"Cart: Add Item"`, `"Search: Apply Filter"`, `"Checkout: Submit Order"`

This approach produces clean, groupable action names that work well in both DQL queries and Dynatrace dashboards.

Renaming Actions in Dynatrace

If auto-detected action names are not descriptive enough, you can configure **user action naming rules** in Dynatrace:

1. Open the mobile application's settings and go to **Naming rules** (RUM Classic: **Frontend** → your app → **Edit** → **Naming rules**)
2. Define a **naming rule** (conditions on the generated name → a fixed name) or an **extraction rule**
3. Test rules against recent action data before deploying

These are mobile rules over the generated action name — not the CSS-selector or page-group rules used for web applications. Check whether your frontend offers the same settings in the New RUM Experience before you rely on them. In code, you can also rename autogenerated actions with the SDK (`DTXAction` modify APIs on iOS).

> **Sources:** User action naming rules for mobile (DT docs) — *"You can define user action naming rules to change the automatically generated names to a predefined name."*; OneAgent SDK for iOS (DT docs) — *"Set naming rules (mobile app settings > Naming rules) to configure user action naming rules or extraction rules"*.
>
> **Important:** Naming rule changes apply only to newly captured actions. Historical data retains the original action names.

Summary

In this notebook, you learned:

- **What user actions are** — discrete interaction events with name, type, duration, and child events

- **Auto-detection capabilities** — UIKit, Android Views, SwiftUI (`DTSwiftInstrumentor`) and Jetpack Compose (plugin 8.271+) are all auto-instrumented
- **Action lifecycle** — from start through child event association to beacon transmission
- **Rage tap detection** — an automatic frustration signal based on rapid repeated taps
- **Custom action instrumentation** — platform-specific code for iOS, Android, and Flutter
- **DQL queries** — how to retrieve and analyze user action data from Grail `user.events`
- **Naming best practices** — descriptive, stable names without dynamic values or user IDs

---

Next Steps

Continue to **MOBL-06: Crash Reporting & ANR Detection** in the Mobile Monitoring series to explore:
- How crashes and ANRs are captured and reported
- Querying crash events from `user.events`
- Symbolication and crash-rate trends

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.